Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: **Agentic Minds**
b. Team leader name: **Madamanchi Ashok Chowdary**
c. Problem Statement: **Future of Work & Enterprise Productivity** — engineers lose most of their week to the toil around code (tests, reviews, builds, deployments). **Prompt2Prod** turns a plain-English request into a planned, tested, reviewed and deployed change.

Brief about the idea

Prompt2Prod is an autonomous AI dev crew on Google Cloud. A developer describes a change in plain English — typed, spoken, or sent from any tool via REST/webhook — and a team of Gemini agents built with Google ADK ships it end-to-end:

1. **Plan** against the real codebase — steps, files, acceptance criteria, risk (Gemini 3.1 Pro)
2. **Write failing tests first** (TDD) for every acceptance criterion
3. **Implement** with sandboxed file & test tools, iterating until the suite is green
4. **AI code review** — quality score, risk level; blocking issues go back to the Coder
5. **Build** with Cloud Build and deploy a **live preview revision** on Cloud Run
6. **Verify the live preview** with AI-generated smoke tests and Gemini visual QA
7. **Release** automatically (Autopilot) or with one-click approval — and roll back in one click

Every thought, tool call and test run streams live to a Firebase-powered dashboard.

2m 27s

median prompt → production
(Autopilot, measured on real requests)

$0.15

average AI cost per shipped change

1m 51s

median pipeline: plan → tests → code → review → build → preview → verify

10

stages · 4 quality gates · 0 hand-offs

Opportunities

🧭 How is it different?

Copilots stop at code suggestions in the IDE and CI/CD only runs pipelines you hand-write. Prompt2Prod closes the **whole loop** — plan → tests → code → review → build → preview → verify → release — on **real infrastructure** (Cloud Build images, Cloud Run revisions), with an audit trail for every change.

⚙️ How does it solve the problem?

It removes hand-offs and waiting: no review queue, no build babysitting, no deploy ritual. Quality is enforced by default — **tests first, AI review, live verification** — while humans stay in control with approval mode, a preview URL for every change and **instant rollback**.

⭐ USP

- **Test-first, self-healing** agent crew (Google ADK)
- **Live preview URL per change** via Cloud Run revision tags
- **AI smoke tests + multimodal visual QA** on the deployed preview
- **Autopilot gate** + human approval + one-click rollback
- Built-in **DORA metrics** & cost per change
- Requests from **anywhere**: UI, voice, REST, webhooks

List of features offered by the solution

Natural-language intake
Dashboard, voice (Web Speech), developer CLI, REST/webhook intake.

Planner agent
Repo-aware plan, acceptance criteria, risk & effort estimate; declines unsafe asks.

Tests-first
Test Author writes failing pytest tests before any code changes (TDD red).

Coder agent
Sandboxed read/write/edit/search/run-tests tools; iterates to green.

Self-healing
Failures are fed back automatically — up to 3 healing attempts.

AI code review
Score 0-100, risk, issues; blocking issues trigger an automatic fix round.

Preview per change
Cloud Build image → tagged Cloud Run revision with its own URL, 0% traffic.

Live verification
Verifier writes HTTP smoke tests, runs them on the preview and self-corrects bad checks.

Visual QA (multimodal)
Gemini looks at a screenshot of the live preview to confirm the UI change.

Autopilot or approval
Release when all gates pass, or wait for one-click human approval.

Instant rollback
Traffic back to the previous revision + git main reset in seconds.

DORA & cost insights
Deploy frequency, lead time, change-failure rate, MTTR, tokens & $ per change.

Process flow diagram or Use-case diagram

Prompt2Prod — from developer prompt to production

Ten stages, four quality gates, zero hand-offs. Median prompt → production ≈ 2 minutes (measured).

1 · Intake
Dashboard · voice · REST
Firebase-authenticated request, rate-limited, queued in Firestore.

2 · Sandbox
Cloud Run Job
Isolated container per task; repo restored from git bundle.

3 · Plan
Planner · Gemini Pro
Steps, files, acceptance criteria, risk, effort estimate.

4 · Tests first
Test Author
Writes failing pytest tests (TDD red phase).

5 · Implement
Coder
Edits code with sandboxed tools, runs tests, iterates.

6 · Test & heal
Test Runner + Coder
Clean pytest run; failures auto-fixed (≤3 attempts).

7 · AI review
Reviewer · Gemini Pro
Score, risk, issues; blocking issues go back to the Coder.

8 · Build
Cloud Build
Commit → container image in Artifact Registry.

9 · Preview + verify
Deployer · Verifier
Tagged Cloud Run revision; AI smoke tests + Gemini visual QA on the live URL.

Gate 1 · Tests green
Full suite incl. the new TDD tests must pass in the sandbox.

Gate 2 · AI review
Verdict approve, risk not high; one automatic fix round.

Gate 3 · Live checks
All smoke tests pass against the real preview revision.

Gate 4 · Release policy
Autopilot ships when gates 1–3 pass; Approval mode waits for a human.

10 · Released 🚀
100% traffic to the new revision, main fast-forwarded, release recorded (DORA metrics).

Awaiting approval
Preview URL ready to try; one click to approve & release or reject.

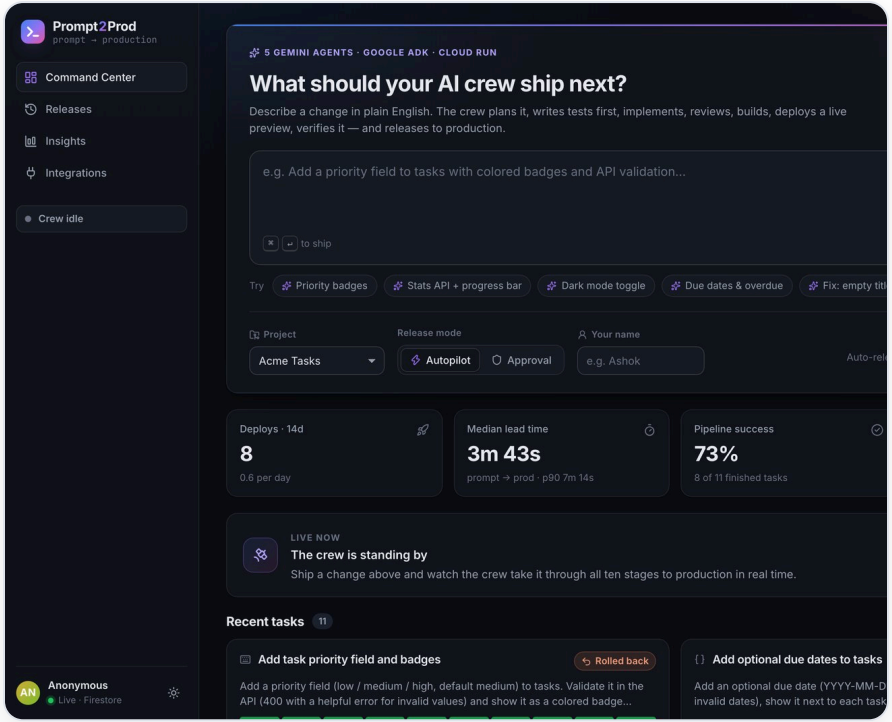
Rollback ↩
One click: traffic back to the previous revision and main reset — production restored in seconds.

⚡ Live: every thought, tool call and test run streams to the dashboard via Firestore

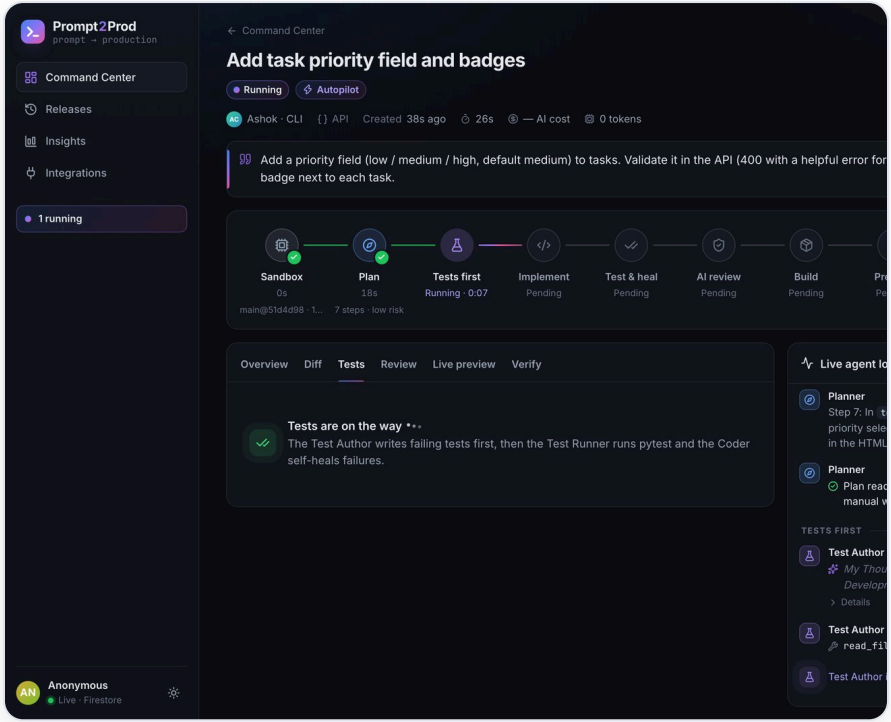
Cost & tokens metered per change

Target app runs with a zero-permission service account

Wireframes/Mock diagrams of the proposed solution (optional)

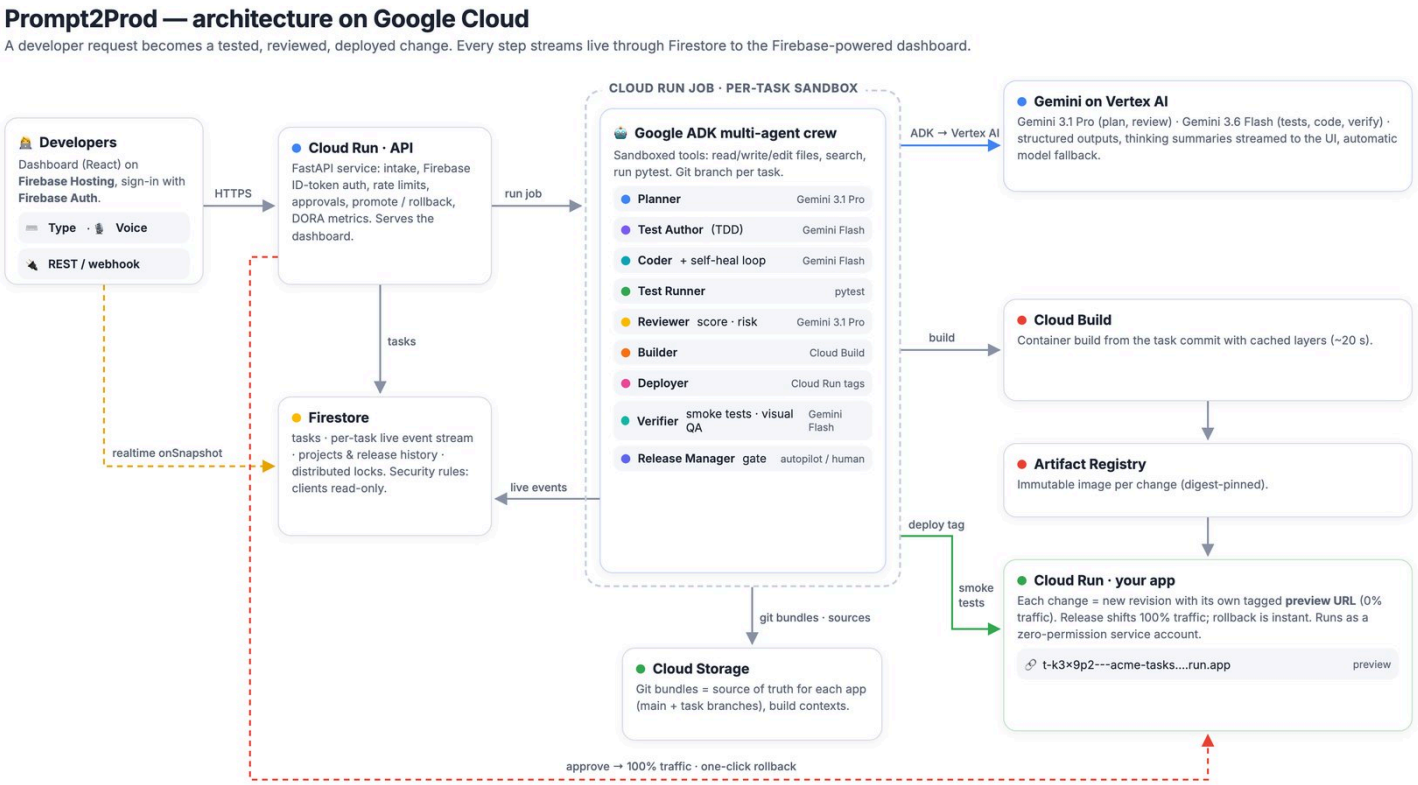


Command Center — compose a request, pick Autopilot or Approval, watch live runs



Task view — 10-stage pipeline, live agent log, plan / diff / tests / review / preview tabs

Architecture diagram of the proposed solution



Technologies to be used in the solution

Gemini on Vertex AI
Gemini 3.1 Pro (Planner, Reviewer) + Gemini 3.6 Flash (Test Author, Coder, Verifier); structured outputs, thinking summaries, model fallback.

Google ADK
Multi-agent crew of LlmAgents with function tools, output schemas and per-role thinking levels.

Cloud Run (service)
FastAPI API + dashboard; Firebase ID-token auth, approvals, releases, metrics.

Cloud Run Jobs
One isolated sandbox execution per request; runs the agents, git and pytest.

Cloud Run revisions & tags
Preview URL per change, traffic switch for release, instant rollback.

Cloud Build + Artifact Registry
Container image per change with layer caching (~20 s builds).

Firestore
Tasks, live per-task event stream, release history, distributed locks.

Firebase
Auth (anonymous sign-in), Hosting (prompt2prod.web.app), JS SDK realtime listeners.

Cloud Storage
Git bundles as source of truth for each app + build contexts.

IAM & Logging
Least-privilege service accounts; target app runs with zero permissions.

App stack
Python 3.12, FastAPI, pytest, git · React + Vite dashboard.

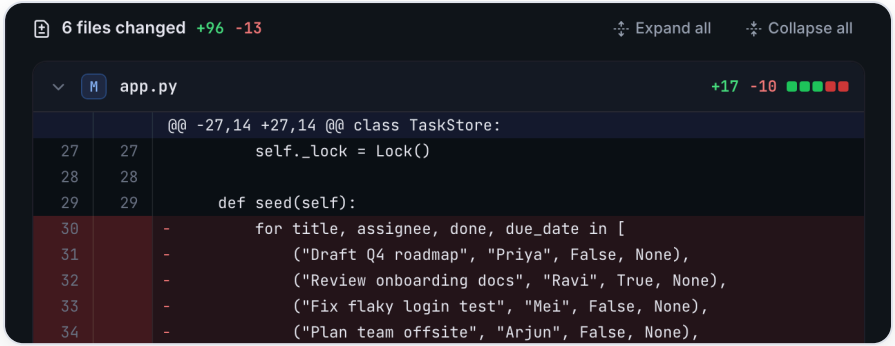
Estimated implementation cost (optional)

Per shipped change (measured)	Cost
Gemini tokens (avg 169K tokens, 18 LLM calls)	$0.15
Cloud Build (~20 s, cached layers)	≈ $0.001
Cloud Run Job sandbox (2 vCPU · 2 GiB · ~2 min)	≈ $0.006
Preview revision (scale-to-zero) · Firestore · Storage	< $0.001
Total per change	**≈ $0.16**

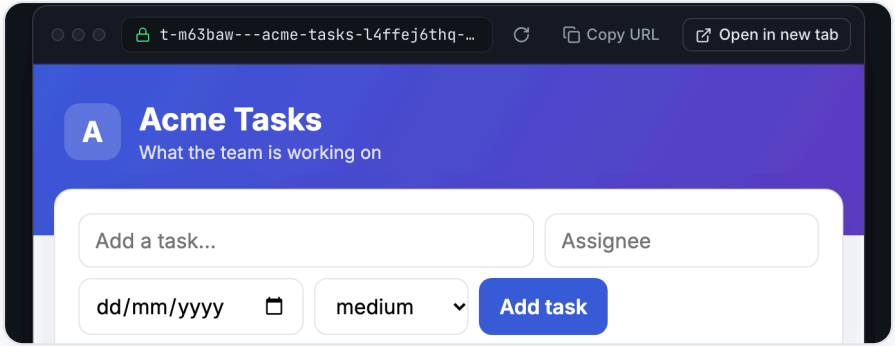
Team of 25 engineers · 400 changes / month	/ month
AI + build + sandbox	≈ $63
API + dashboard (1 warm Cloud Run instance)	≈ $30
Firestore · Storage · Artifact Registry	≈ $5
Platform total	**≈ $98**

vs. ~2.1 engineer-hours estimated per change by the Planner → hundreds of hours returned to the team every month. Prototype runs fully on the $300 Google Cloud trial.

Snapshots of the prototype



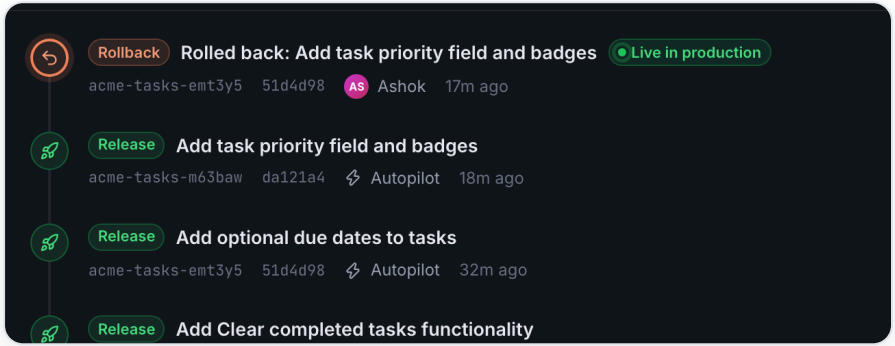
AI-written diff (6 files, +96 –13), tested, reviewed 100/100 and committed



Live preview: the tagged Cloud Run revision embedded in the task view



Insights — DORA & AI-productivity metrics computed from real runs



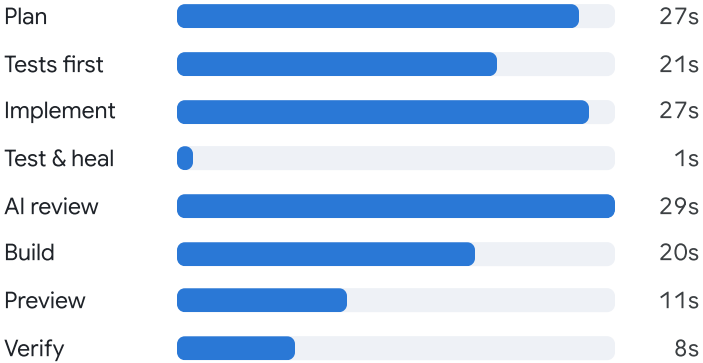
Releases — autopilot & human-approved releases, rollbacks, revisions, commits

Prototype Performance report/Benchmarking

Real request (Acme Tasks app)	Pipeline	Files	Tests (new)	Review	Live checks	AI cost
Reject empty task titles with 400 error	1m 24s	2 (+9)	13/13 (+1)	100	5/5	$0.05
Add assignee filtering to tasks API	1m 27s	2 (+26)	12/12 (+1)	100	4/4	$0.05
Add client-side task search filter	1m 30s	4 (+55)	13/13 (+0)	100	4/4	$0.09
Add Clear completed tasks functionality	1m 41s	6 (+63)	15/15 (+2)	100	5/5	$0.12
Add stats API and task completion progress	2m 01s	6 (+62)	11/11 (+2)	100	3/3	$0.11
Add task priority field and badges	2m 18s	6 (+96)	24/24 (+5)	100	5/5	$0.18
Add task priority field and badges	2m 51s	6 (+85)	23/23 (+4)	100	4/4	$0.22

9 measured end-to-end runs · median pipeline 1m 51s · median Autopilot lead time 2m 27s (excluding time waiting in the queue) · every run ended with a green suite and a 95–100 AI review. Tuning per-agent models & thinking levels cut the first run from 5m 15s to ~1.5 min; three early runs paused at the live-check gate because of a verifier bug (now self-correcting) and were re-run.

Average seconds per stage



Real catch, task m5cahc: the AI Reviewer flagged a CSS selector typo (`.new-task` → `.new_task`) that would have broken the form styling — invisible to every unit test. The Coder fixed it automatically, the re-review approved, and Autopilot shipped it in 3m 03s.

Additional Details/Future Development (if any)

🔌 Connect real repos

GitHub/GitLab apps: issue → PR with preview link, review comments, merge on approval. Monorepos & multi-service changes.

🧩 Any stack

Buildpacks + language packs (Node, Go, Java); Agent Engine for long-running, resumable agent sessions; codebase RAG with Vertex AI Search for large repos.

🚦 Progressive delivery

Canary traffic splits with Cloud Monitoring SLOs and automatic rollback; feature flags; scheduled releases.

🛡️ Enterprise guardrails

Policy-as-code (protected files, budgets), per-tenant least-privilege sandboxes, VPC-SC, SSO via Identity Platform, audit export to BigQuery.

💬 Team copilots

Slack / Google Chat intake, standup summaries of shipped work, release notes generated from commits.

🧠 Learning loop

Agents learn from review feedback and rollbacks; org-wide conventions captured automatically in AGENTS.md.

Provide links to your:

GitHub Public Repository

https://github.com/ashok2605/prompt2prod

Demo Video Link (3 Minutes)

https://youtu.be/7gPnEYwUhyk

Final Product Link

https://prompt2prod.web.app

Cloud Run: https://prompt2prod-82783179546.asia-south1.run.app · Demo app in production: https://acme-tasks-82783179546.asia-south1.run.app

Security, guardrails & responsible AI

🔒 Isolation

Every request runs in its own Cloud Run Job container (gVisor sandbox) on a fresh git branch; agents can only touch files inside the repo.

🪪 Least privilege

The deployed app runs as a service account with zero IAM roles. Dashboard clients get read-only Firestore access via security rules.

🙋 Human in the loop

Approval mode, preview URL per change, reject at any time, one-click rollback; autopilot only ships when tests, review and live checks all pass.

🎯 Scope control

The Planner declines unsafe or out-of-scope requests; the Test Author may only edit tests/; reviewers flag security issues.

💸 Abuse & cost limits

Firebase-authenticated intake, per-user and daily rate limits, per-run LLM call caps, cost metering per change.

🧾 Full audit trail

Plan, prompts' outcome, diff, tests, review, build, release and rollback events are stored per task in Firestore and git.

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

